

Random Forest: From First Principles

Why Do We Need Random Forest?

A single Decision Tree is easy to understand, but it has one serious weakness: it can become too sensitive to the training data. If the data changes slightly, the tree may choose a different root split, and once the root changes, the whole tree below it can change. This means one tree can be unstable.

Imagine asking one doctor for a diagnosis. If the doctor is excellent, the answer may be useful. But one doctor can still make a mistake, especially if the case is unusual. Now imagine asking many doctors independently and then combining their opinions. One doctor may overfocus on one symptom, another may notice a different pattern, and another may be wrong. But when many independent opinions are combined, the final decision is usually more reliable.

Random Forest follows this same idea. Instead of trusting one Decision Tree, it builds many Decision Trees and combines their predictions. Each tree is trained slightly differently, so the trees do not all make the exact same mistakes. The final prediction becomes more stable than the prediction of a single tree.

The main reason Random Forest exists is to reduce the weakness of a single Decision Tree while keeping the useful parts of trees. It still uses tree-based questions, so it can handle non-linear patterns. But by combining many trees, it reduces overfitting and high variance.

Problems with a Single Decision Tree

A single Decision Tree can overfit the training data. If the tree is allowed to grow very deep, it may create very specific rules that work perfectly on training examples but fail on new examples.

For example, a tree predicting loan approval might learn a rule like: if income is exactly around 58200, age is 37, city is a certain value, and credit history has a specific pattern, approve the loan. This rule may match a few training examples, but it may not represent a real general pattern.

A single tree also has high variance. High variance means the model changes a lot when the training data changes slightly. If a few rows are added or removed, the best split at the root may change. Once the root split changes, many later splits can also change.

This instability is not because Decision Trees are useless. It happens because trees are greedy. A tree chooses the best split at the current node, then builds the rest of the tree based on that decision. Early decisions strongly affect later decisions.

Random Forest was created to solve this problem. Instead of building one deep tree and trusting it completely, Random Forest builds many trees and combines them. The mistakes of one tree can be balanced by the other trees.

What Is a Random Forest?

A Random Forest is a machine learning algorithm made of many Decision Trees.

Each tree is trained on a slightly different version of the training data. Each tree also gets some randomness while choosing features for splits. Because of this, the trees become different from each other.

After all trees are trained, Random Forest combines their predictions.

For classification, the trees vote for the class. The class with the most votes becomes the final prediction.

For regression, the trees predict numbers. The final prediction is usually the average of all tree predictions.

The word forest is used because the model contains many trees. The word random is used because the trees are built using randomness in data sampling and feature selection.

Random Forest is an ensemble method. An ensemble method combines multiple models to create a stronger model. In Random Forest, the individual models are Decision Trees. Voting and averaging are ensemble techniques, but they can be studied more deeply in a separate Ensemble Learning topic. For Random Forest, the most important idea is simple: many trees working together are usually more reliable than one tree alone.

Bootstrap Sampling

Bootstrap sampling is one of the main sources of randomness in Random Forest.

Suppose we have a training dataset with 1000 rows. Instead of giving every tree the exact same 1000 rows, Random Forest creates a different sample for each tree.

A bootstrap sample is created by sampling with replacement. Sampling with replacement means that after a row is selected, it is placed back into the dataset and can be selected again.

Because of this, one bootstrap sample may contain the same row multiple times, and some original rows may not appear at all.

For example, suppose the original data is:

Row	Value
1	A
2	B
3	C
4	D
5	E

One bootstrap sample might be:

Selected Row
A
C
C
E
B

Notice that C appeared twice and D did not appear at all. That is normal.

This helps Random Forest because each tree sees a slightly different dataset. If every tree saw the exact same data and used the exact same features, all trees might become very similar. Bootstrap sampling encourages diversity among trees.

Diversity is important. If all trees make the same mistake, combining them does not help much. If different trees make different mistakes, combining them can reduce the final error.

Random Feature Selection

Bootstrap sampling makes the data different for each tree. Random feature selection makes the split choices different inside each tree.

In a normal Decision Tree, when the tree searches for the best split, it can look at all features. Random Forest changes this. At each split, the tree only looks at a random subset of features.

Suppose a dataset has 20 features. At one node, a tree may only be allowed to consider 5 randomly chosen features. At another node, it may consider a different set of 5 features.

This prevents one very strong feature from dominating every tree. If all trees always used the same strongest feature at the root, the trees would become similar. Random feature selection forces trees to explore different patterns.

For classification, a common choice is to consider about $\sqrt{d}$ features at each split, where d is the total number of features.

$$\text{Number of features considered per split} \approx \sqrt{d}$$

For regression, a common choice may be a larger fraction of the features, depending on the implementation and problem.

The exact number is a hyperparameter. The intuition is more important than memorizing the rule: each tree should be strong enough to learn useful patterns, but different enough from other trees to reduce shared mistakes.

Building Multiple Trees

Random Forest builds many Decision Trees. Each tree is trained independently.

The process looks like this:

Step	What Happens
1	Create a bootstrap sample from the training data.
2	Train a Decision Tree on that sample.
3	At each split, consider only a random subset of features.
4	Let the tree grow, usually without pruning or with limited controls.
5	Repeat the process many times to build many trees.

Each tree learns rules from its own sampled data and random feature choices. Some trees may become very good in one region of the data. Other trees may be better in another region.

The forest becomes powerful because it combines many imperfect trees. A single tree may be noisy, but the average behavior of many trees is more stable.

This is the central mechanism of Random Forest: build many different trees, then combine their answers.

Classification with Majority Voting

For classification, each tree predicts a class. The forest then uses majority voting.

Suppose we want to predict whether an email is Spam or Not Spam. Imagine a forest with 7 trees.

Tree	Prediction
Tree 1	Spam
Tree 2	Spam
Tree 3	Not Spam
Tree 4	Spam
Tree 5	Not Spam
Tree 6	Spam
Tree 7	Spam

Five trees predicted Spam. Two trees predicted Not Spam. The final Random Forest prediction is Spam.

Majority voting means the class with the most votes wins.

$$\textit{Final Class} = \textit{majority vote}(T_1(x), T_2(x), \dots, T_B(x))$$

Here, B is the number of trees in the forest. The expression $T_1(x)$ means the prediction made by the first tree for input x .

Voting is an ensemble technique. It will make even more sense when studying Ensemble Learning separately. For Random Forest, just remember that classification trees vote, and the forest trusts the majority decision.

Majority voting reduces the effect of one bad tree. If one tree makes a mistake, many other trees can still outvote it.

Regression with Averaging

For regression, each tree predicts a number. The forest then averages the predictions.

Suppose we want to predict house price. A forest with 5 trees gives these predictions:

Tree	Predicted Price
Tree 1	300000
Tree 2	320000
Tree 3	310000
Tree 4	305000
Tree 5	315000

The final prediction is the average:

$$\textit{Final Prediction} = \frac{300000 + 320000 + 310000 + 305000 + 315000}{5}$$

$$\textit{Final Prediction} = 310000$$

The general formula is:

$$\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$$

Here, B is the number of trees. The term $T_b(x)$ means the prediction made by tree b for input x . The symbol $\sum$ means we add the predictions from all trees.

Averaging is also an ensemble method. The deeper theory belongs in Ensemble Learning, but the Random Forest idea is simple: many noisy numerical predictions are averaged to create a smoother and more stable final prediction.

Out-of-Bag Error

Out-of-Bag Error is one of the most elegant ideas in Random Forest.

Because each tree is trained on a bootstrap sample, some training rows are not selected for that tree. These unused rows are called out-of-bag samples for that tree.

For each tree, the data can be divided into two groups:

Data Type	Meaning
In-bag data	Rows selected in the bootstrap sample and used to train the tree.
Out-of-bag data	Rows not selected for that tree.

The out-of-bag rows can be used like a small validation set for that tree. The tree did not train on those rows, so testing on them gives a useful estimate of performance.

Across the whole forest, every training row is out-of-bag for some trees. Random Forest can collect predictions from only the trees that did not train on that row. Then it compares those predictions with the true answer.

This gives the Out-of-Bag Error.

A common fact is that each bootstrap sample leaves out about 36.8% of the original rows for a given tree. The reason comes from probability.

For one row in a dataset of size n , the probability that it is not selected in one draw is:

$$1 - \frac{1}{n}$$

A bootstrap sample makes n draws. So the probability that the row is never selected is:

$$\left(1 - \frac{1}{n}\right)^n$$

For large n , this approaches:

$$\frac{1}{e} \approx 0.368$$

So about 36.8% of rows are out-of-bag for each tree.

OOB Error is useful because it estimates model performance without needing a separate validation set. It is not magic, and a separate test set is still important for final evaluation, but OOB Error is a very helpful internal estimate.

Feature Importance

Random Forest can estimate which features are important.

The most common idea is based on impurity reduction. If a feature is used often in splits that greatly reduce impurity, it receives higher importance.

Suppose a Random Forest predicts loan approval. If Credit Score repeatedly creates strong splits across many trees, Credit Score will get high importance. If Favorite Color almost never improves the splits, it will get low importance.

Feature importance is usually averaged across all trees. This makes it more stable than feature importance from a single Decision Tree.

A simple way to think about it is:

Feature Importance = average impurity reduction caused by that feature

Feature importance is helpful, but it should not be treated as perfect truth. Features with many possible split points can sometimes look more important. Correlated features can also share importance in confusing ways. If two features carry similar information, the forest may use one more often and make the other look less important.

Feature importance tells us which features the forest used most effectively, not necessarily which features are truly causal in the real world.

Important Hyperparameters

Hyperparameters control how the forest is built.

The `n_estimators` hyperparameter controls the number of trees. More trees usually make predictions more stable, but they also increase training time, prediction time, and memory use.

The `max_depth` hyperparameter controls how deep each tree can grow. Deeper trees can learn more complex patterns, but they can also overfit.

The `min_samples_split` hyperparameter controls the minimum number of samples required to split a node. Larger values make trees more conservative.

The `min_samples_leaf` hyperparameter controls the minimum number of samples allowed in each leaf. Increasing this value can reduce overfitting because leaves cannot become too specific.

The `max_features` hyperparameter controls how many features are considered at each split. This is one of the most important Random Forest settings because it controls how different the trees become.

The `bootstrap` hyperparameter controls whether bootstrap sampling is used. Random Forest usually uses bootstrap sampling because it helps create different trees and makes OOB Error possible.

The `oob_score` setting controls whether out-of-bag performance is calculated.

The `criterion` controls how each tree measures split quality. For classification, common choices are Gini and Entropy. For regression, common choices are based on squared error or absolute error.

The main tuning idea is balance. We want trees to be strong enough to learn useful patterns, but different enough from one another so that combining them reduces variance.

Computational Complexity

Random Forest is more expensive than a single Decision Tree because it builds many trees.

Let n be the number of training examples, d be the number of features, B be the number of trees, and h be the average depth of the trees.

Training one Decision Tree requires searching for useful splits. Random Forest repeats that process B times.

A simplified training complexity is:

$$O(B \cdot n \cdot m \cdot \log n)$$

Here, m is the number of features considered at each split. Usually m is smaller than d because Random Forest uses random feature selection.

Training is slower because the forest must build many trees, and each tree must search for good splits at many nodes.

Prediction is faster than training, but slower than one single tree. For one new sample, the sample must pass through all B trees. If the average tree depth is h , prediction cost is approximately:

$$O(B \cdot h)$$

Prediction is still usually fast because each tree only follows one path from root to leaf. The model does not search for new splits during prediction. The structure is already learned.

Memory usage depends on the number of trees and the size of each tree.

$$\text{Memory} \approx O(B \cdot \text{number of nodes per tree})$$

More trees usually improve stability, but they require more memory.

The difference between training and prediction can be understood like this:

Training	Prediction
Builds many trees	Uses already-built trees
Searches many split points	Follows one path per tree
Measures impurity repeatedly	Reads stored split rules
Slower because learning happens here	Faster because only decision paths are followed

Training is slower because the forest is discovering structure. Prediction is faster because the forest is only using structure that already exists.

Advantages of Random Forest

Random Forest usually performs better than a single Decision Tree because it reduces variance. One tree may overfit, but many trees averaged together become more stable.

It works well for both classification and regression problems.

It can capture non-linear relationships because each tree can create complex decision boundaries.

It does not require feature scaling. Features do not need to be standardized or normalized because trees split using thresholds.

It can handle many features and often works well even when the relationship between features is complicated.

It gives feature importance, which can help us understand which variables are useful for prediction.

It is less sensitive to noise than a single Decision Tree because one noisy tree does not control the whole prediction.

It can use Out-of-Bag Error as an internal performance estimate.

Disadvantages of Random Forest

Random Forest is less interpretable than a single Decision Tree. One tree can be drawn and explained. A forest with hundreds of trees is much harder to explain directly.

It uses more memory because it stores many trees instead of one.

It is slower to train than a single Decision Tree because many trees must be built.

It is slower to predict than a single Decision Tree because every sample must pass through many trees.

It may not perform as well as boosting methods on some structured/tabular problems where careful boosting can reduce bias more effectively.

Feature importance can be misleading when features are correlated or when some features have many possible split points.

Random Forest also does not naturally extrapolate well in regression. Because tree predictions are based on averages in leaf nodes, a Random Forest usually cannot predict far beyond the range of values seen in training data.

Evaluating Random Forest

Evaluation depends on the type of problem.

For classification, Random Forest can be evaluated using Confusion Matrix, Accuracy, Precision, Recall, F1-Score, and ROC-AUC.

Accuracy measures the percentage of correct predictions. It is useful when classes are balanced.

Precision measures how often positive predictions are actually correct. It is important when false positives are costly.

Recall measures how many actual positives the model successfully finds. It is important when false negatives are costly.

F1-Score balances Precision and Recall.

ROC-AUC measures how well the model separates classes across different thresholds.

For regression, Random Forest can be evaluated using MAE, MSE, RMSE, and R^2 .

MAE gives the average absolute error. MSE gives stronger punishment to large errors. RMSE brings the error back to the target's original unit. R^2 compares the model against predicting the average value.

Out-of-Bag Error can also be used as an internal estimate of performance. It is especially useful during training because it uses the samples left out of each tree's bootstrap sample.

Even when OOB Error is available, a final test set is still important. The test set gives the cleanest estimate of how the model performs on unseen data.

How Random Forest Fits into Ensemble Learning

Random Forest is an ensemble algorithm because it combines many models.

More specifically, Random Forest is based on bagging plus random feature selection.

Bagging means building many models on different bootstrap samples and combining their predictions. Random Forest adds another layer of randomness by limiting the features considered at each split.

This relationship can be shown as:

Concept	Meaning in Random Forest
Bagging	Each tree trains on a bootstrap sample.
Random Feature Selection	Each split considers only a random subset of features.
Voting	Classification trees vote for the final class.
Averaging	Regression trees average their predictions.

Other ensemble algorithms use different ideas. AdaBoost, Gradient Boosting, XGBoost, LightGBM, and CatBoost are mainly boosting methods. Boosting builds models sequentially, where later models try to correct earlier mistakes.

Random Forest is different. Its trees are usually built independently. The power comes from diversity and averaging, not from correcting mistakes step by step.

This is why Random Forest is often easier to understand before studying boosting algorithms. It shows the basic power of combining many trees while keeping the learning process relatively intuitive.

When to Use Random Forest and When Not to Use It

Use Random Forest when you want a strong model for tabular data without needing heavy preprocessing.

Use it when a single Decision Tree overfits but tree-based logic still seems useful.

Use it when the relationship between features and target is non-linear or involves feature interactions.

Use it when you want a reliable baseline model that usually performs well.

Use it when feature scaling would be inconvenient, because Random Forest does not require scaling.

Use it when feature importance is useful for understanding the model.

Be careful using Random Forest when interpretability is the highest priority. A single Decision Tree is easier to explain than a forest.

Be careful when memory or prediction speed is limited. A large forest can become heavy.

Be careful for regression problems that require extrapolation beyond the training range. Random Forest is usually not good at predicting values far outside what it has already seen.

Avoid assuming Random Forest is always the best tree-based model. Boosting methods may perform better in many competitions and production tabular problems, but they are also more sensitive to tuning.

Random Forest is best understood as a powerful improvement over a single Decision Tree. It keeps the flexibility of trees, reduces their instability, and prepares the foundation for understanding more advanced ensemble models.